

## Activity 3: Collaborative Coding with GitLens and Live Share

|                    |                                                                         |                 |                                     |
|--------------------|-------------------------------------------------------------------------|-----------------|-------------------------------------|
| <b>Course Code</b> | B25GE0101                                                               | <b>Session</b>  | Activity 3                          |
| <b>Course</b>      | Portfolio Building for Engineering Students — GitHub-Integrated Edition | <b>Duration</b> | 2 hours (studio)                    |
| <b>Semester</b>    | 3rd Sem CSE                                                             | <b>Mode</b>     | Pair activity (2 students per pair) |

### Objective

By the end of this activity, students will be able to:

- Use the GitLens extension in VS Code to inspect commit history, file blame, and line-by-line change authorship.
- Use the Live Share extension to start and join a real-time collaborative coding session.
- Collaboratively write, review, and commit code with a peer, then verify authorship and history using GitLens.

### Prerequisites

Completed from earlier activities:

- Activity 1: VS Code, C/C++ extension, and MinGW-w64/GCC installed and working.
- Activity 2: GitHub account created, Git configured, and the Hello World repository cloned and pushed with a README.

A pair of students working together, each with their own laptop and their own copy (clone) of their partner's repository, or one shared repository both are collaborators on.

### Tools Used

|                   |                                                                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>GitLens</b>    | VS Code extension that adds Git blame annotations, commit history, and file/line history views directly in the editor.          |
| <b>Live Share</b> | VS Code extension (Microsoft) that lets one developer host a session others can join to co-edit and co-debug code in real time. |

### Part A — Installing and Exploring GitLens

1. Open VS Code. Go to the Extensions view (Ctrl+Shift+X).
2. Search for "GitLens — Git supercharged" by GitKraken and click Install.
3. Open your Hello World project folder from Activity 2 (the one already pushed to GitHub).
4. Open hello.c. Notice the faint grey text at the end of each line — this is GitLens blame, showing who last changed that line and when.

5. Click the GitLens icon in the Activity Bar (left sidebar) to open the GitLens view. Expand "Commits" to see the full commit history of the repository.
6. Right-click hello.c in the Explorer and choose "Open File History" to see every commit that touched this file.
7. Hover over any blame annotation to see the full commit message, author, and date in a popup.

## Part B — Installing and Starting Live Share

8. In Extensions, search for "Live Share" by Microsoft and click Install. Sign in with a Microsoft or GitHub account when prompted.
9. One partner (the Host) opens the Hello World project, opens the Command Palette (Ctrl+Shift+P), and runs "Live Share: Start Collaboration Session".
10. VS Code copies a session link to the clipboard. The Host shares this link with their partner (chat, email, etc.).
11. The other partner (the Guest) opens the Command Palette and runs "Live Share: Join Collaboration Session", pasting the link.
12. Once connected, both partners see the same files. The Guest's cursor appears in the Host's editor with their name label, and vice versa.

## Part C — Paired Task: Extend Hello World Together

Working inside the same Live Share session, complete the following as a pair:

13. The Host opens hello.c. Together, decide who types first.
14. Add a new function to the program that greets a person by name, for example:

```
void greet(const char *name) {  
    printf("Hello, %s! Welcome to your GitHub portfolio.\n", name);  
}
```

15. Call this function from main() with a sample name, e.g. greet("Ada");
16. Swap control: use the Live Share "Follow" and "Focus Participants" options so the Guest can drive for a few minutes while the Host observes, then switch back.
17. The Guest uses the Live Share integrated terminal (shared with the Host) to compile and run the program on the Host's machine, confirming the greeting prints correctly.
18. End the session: the Host runs "Live Share: End Collaboration Session" from the Command Palette.

## Part D — Commit, Push, and Verify with GitLens

19. On the Host's machine, stage and commit the change:

```
git add hello.c  
git commit -m "Add greet() function — paired with <partner name> via Live Share"
```

20. Push the commit to GitHub:

```
git push origin main
```

21. Open GitLens' "Commits" view again and confirm the new commit appears at the top with the correct message and timestamp.
22. Hover over the blame annotation on the new greet() lines to confirm GitLens attributes them to this commit.

23. On GitHub, open the repository in the browser and check that the commit and the updated README (if edited) are visible in the commit history.

## **Deliverable — Portfolio Entry**

Update the project README with a short "Collaboration Log" section noting:

- The name of the pairing partner and their GitHub username.
- What was built together (the greet() function).
- One thing learned about using GitLens or Live Share.

Commit this README update as a second, separate commit, then push it.

## **Reflection Questions**

- How did GitLens' blame view help you understand who changed what, and when, in code you did not write yourself?
- What was one advantage and one limitation of pair programming through Live Share compared to sitting next to your partner?
- Why might employers value seeing evidence of collaborative commits in a portfolio repository?

---

*Faculty note: circulate during Part C to confirm every pair has swapped driver/navigator roles at least once before Part D.*